

TRABAJO PRÁCTICO INTEGRADOR

AUTOMATIZACIÓN DE PROCESOS ADMINISTRATIVOS:

CHATBOT DE GESTION DE HORAS EXTRAS.

Alumno:

Mariano Avendaño

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

Organización Empresarial

Docente Titular

Prof. Gabriela Martínez

Docentes Adjuntos

Prof. Carolina Bruno / Prof. Mario Raúl López / Prof. Andrea Ramos

25 de Junio de 2026

Tabla de contenido

Introducción	3
Objetivos	3
Consignas	3
Desarrollo	4
Modelado de negocio y compuertas bpmn 2.0	4
Arquitectura conversacional	5
Diccionario de datos (estructura csv)	6
Manual de usuario	8
Pruebas de caminos infelices (robustez)	9
Herramientas de IA utilizadas	10
Conclusión	11
Referencias	11

Introducción

El presente trabajo práctico integrador aborda el diseño, modelado en BPMN 2.0 y desarrollo de un chatbot real en Telegram para optimizar la Gestión de Horas Extras y la Movilidad del personal. El sistema incluye tres actores diferenciados: el Empleado que solicita turnos, el Chatbot que automatiza las validaciones y registros, y el Supervisor que aprueba o rechaza las solicitudes. Los datos se gestionan mediante archivos CSV que actúan como base de datos transaccional.

Objetivos

Que los participantes logren:

- Modelar un proceso administrativo optimizado mediante compuertas lógicas empleando la notación estándar BPMN 2.0 con tres carriles (Empleado, Sistema, Supervisor).
- Desarrollar un chatbot conversacional real integrado con la API de bots de Telegram con una máquina de estados sólida en Python.
- Garantizar la robustez del sistema validando las entradas del usuario (camino infeliz) y controlando la persistencia de datos mediante archivos CSV.

Desarrollo

1. Modelado de negocio y compuertas bpmn 2.0

El proceso de negocio se estructuró en tres carriles (Lanes) según la notación BPMN 2.0:

- Empleado: Inicia el proceso, selecciona opciones del menú, elige fechas, confirma cancelaciones y gestiona transporte.
- Chatbot / Sistema: Lee y escribe archivos CSV, valida fechas disponibles, registra solicitudes y notifica resultados.
- Supervisor: Revisa solicitudes pendientes, aprueba o rechaza turnos y cancelaciones, y carga nuevas fechas disponibles.

El modelo contempla 6 compuertas lógicas (Gateways) implementadas en el código:

1. ¿Hay fechas disponibles?: Valida en la base de datos si existen fechas planificadas en `fechas_disponibles.csv`.
2. ¿Existen registros históricos?: Verifica si el empleado posee solicitudes previas al consultar el Historial.
3. ¿Tiene solicitudes activas?: Controla que existan solicitudes modificables antes de ofrecer la cancelación.
4. ¿Opción de fecha válida?: Valida que el número ingresado corresponda a una fecha existente en el listado.

5. ¿Solicita movilidad o movilidad propia?: Bifurca el subflujo de Transporte entre vehículo provisto o traslado por sus propios medios.

6. ¿Dirección válida?: Valida que el domicilio tenga al menos 5 caracteres y contenga un número de altura.

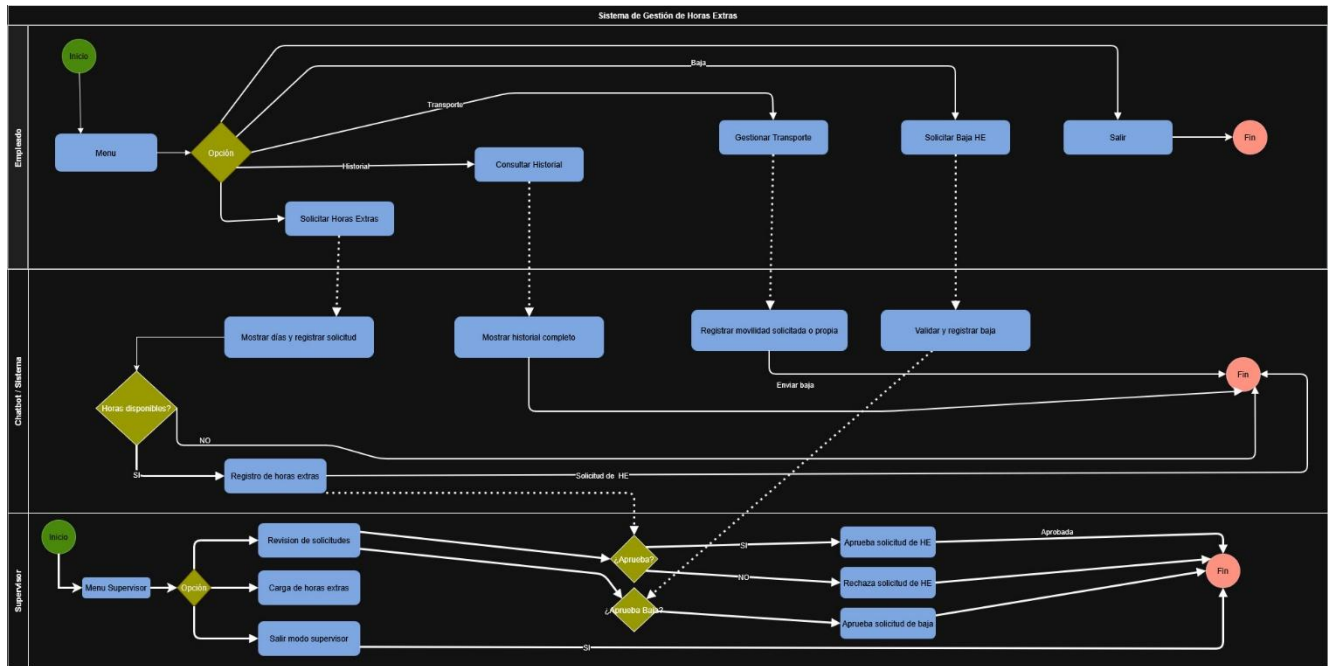


Figura 1: Modelado de Procesos BPMN 2.0 - Chatbot Telegram Horas Extras

2. Arquitectura conversacional (máquina de estados)

El chatbot está gobernado por una Máquina de Estados Finita (FSM) que memoriza el paso actual de cada conversación por chat_id. El bot se integra de forma directa con la API oficial de Telegram realizando HTTP Polling. La lógica se desarrolla en Python bajo la siguiente estructura de estados:

- INACTIVO: Esperando mensaje de inicio (/start, hola, menu).
- MENU: Esperando selección del menú principal (1 al 5).
- ELIGIENDO_FECHA: Esperando que el usuario elija una de las fechas disponibles (ej. 1, 2 o 3).

- ELIGIENDO_BAJA: Esperando que el usuario seleccione qué solicitud cancelar.
- CONFIRMANDO_BAJA: Esperando confirmación (1 = Sí, 2 = No) para la cancelación.
- TRANSPORTE_TIPO: Esperando selección entre movilidad provista (1) o propia (2).
- TRANSPORTE_DIRECCION: Esperando el ingreso del domicilio y validando su formato.
- SUPERVISOR: Menú principal del supervisor con solicitudes pendientes.
- SUPERVISOR_REVISANDO: Evaluación para aprobar o rechazar una solicitud.
- SUPERVISOR_NUEVA_FECHA / SUPERVISOR_NUEVA_FECHA_LABEL: Ingreso de nueva fecha disponible en el sistema.

3. Diccionario de datos (estructura csv)

La persistencia de información se implementa en archivos planos CSV locales, garantizando la simplicidad del desarrollo:

Tabla 1: Fechas Disponibles (fechas_disponibles.csv)

Campo	Tipo de Datos	Descripción	Ejemplo
id	String	ID de fecha habilitada	DT001
fecha	String	Fecha exacta (AAAA-MM-DD)	2026-07-04
label	String	Etiqueta para menú del bot	Sábado 04/07

Tabla 2: Solicitudes de Horas (solicitudes_horas.csv)

Campo	Tipo de Datos	Descripción	Ejemplo
id	String	ID único de solicitud	REC001
usuario_id	String	ID de chat de Telegram del empleado	123456789
usuario_nombre	String	Nombre registrado en Telegram	Mariano Avendaño
fecha	String	Fecha del turno (AAAA-MM-DD)	2026-07-04
label_fecha	String	Etiqueta del día	Sábado 04/07
horas	Entero	Cantidad de horas asignadas	4
estado	String	Estado de la solicitud	Solicitada / Aprobada / Rechazada / Baja Solicitada
supervisor	String	Estado de revisión del supervisor	Pendiente / Aprobado / Rechazado

Tabla 3: Solicitudes de Transporte (solicitudes_transporte.csv)

Campo	Tipo de Datos	Descripción	Ejemplo
id	String	ID único de vale de transporte	TRP001
usuario_id	String	ID de chat de Telegram del empleado	123456789
usuario_nombre	String	Nombre del pasajero	Mariano Avendaño
fecha	String	Fecha del servicio (AAAA-MM-DD)	2026-06-25
tipo	String	Tipo de traslado	Movilidad propia / Solicitar movilidad
direccion	String	Dirección de destino de domicilio	Av. Corrientes 1234
timestamp	String	Timestamp de registro (ISO)	2026-06-25T20:30:00Z

4. MANUAL DE USUARIO

Para iniciar y probar la conversación del bot en Telegram:

1. Iniciar chat: Abra Telegram, busque su bot por su @username e inicie la conversación enviando '/start' o 'hola'.
2. Menú Principal: El bot mostrará 5 opciones numeradas.
3. Solicitar Horas Extras (Opción 1): Muestra las fechas disponibles. El empleado elige una y la solicitud queda 'Pendiente' de aprobación del supervisor. La fecha se elimina del listado para que nadie más la seleccione.
4. Consultar Historial (Opción 2): Lista todas las solicitudes del empleado con su estado y la revisión del supervisor.
5. Cancelar solicitud (Opción 3): Muestra las solicitudes activas del empleado. Al confirmar, se registra una 'Baja Solicitada' que el supervisor debe aprobar.
6. Gestionar Transporte (Opción 4): Disponible solo si el empleado tiene horas solicitadas. Permite registrar movilidad de la empresa (ingresando dirección válida) o movilidad propia.
7. Salir (Opción 5): Finaliza la sesión.

Modo Supervisor: Cualquier usuario puede escribir 'supervisor' para acceder al panel. Allí puede:

- Revisar solicitudes pendientes y aprobarlas o rechazarlas.
- Agregar nuevas fechas disponibles al sistema (opción A).
- Salir del modo supervisor (opción X).

5. Pruebas de caminos infelices (robustez)

El bot cuenta con validaciones para reaccionar ante errores del usuario sin interrumpir el servicio:

- Caso 1: Opción inválida en el menú
 - Usuario escribe: 'quiero horas'
 - Bot responde: 'Opcion invalida. Escribi 1, 2, 3, 4 o 5.' y reenvía el menú.
- Caso 2: Texto en lugar de número al elegir fecha
 - Usuario escribe: 'martes'
 - Bot captura ValueError y responde: 'Por favor escribi SOLO el numero'.
- Caso 3: Dirección sin número de altura
 - Usuario escribe: 'mi casa'
 - Bot valida con `tiene_numero()` y responde: 'La direccion debe tener un numero de altura'.
- Caso 4: Sin fechas disponibles
 - Cuando `fechas_disponibles.csv` está vacío, el bot informa: 'No hay horas extras disponibles' y sugiere contactar al supervisor.
- Caso 5: Sin solicitudes activas para cancelar
 - El bot verifica que existan solicitudes modificables antes de ofrecer la cancelación.

- Caso 6: Transporte sin horas solicitadas
 - El bot exige tener al menos una solicitud de horas extras para acceder a la gestión de transporte.

6. Herramientas de IA utilizadas

Para el desarrollo de este trabajo se utilizó OpenCode como agente de codificación autónomo basado en IA. Las principales tareas asistidas fueron:

- Generación inicial del código Python del chatbot con máquina de estados.
- Corrección de errores: reemplazo de método padStart (JavaScript) por zfill (Python), rutas relativas a absolutas, y reparación de CSV corrupto.
- Simplificación del código a un estilo principiante con comentarios explicativos en español.
- Implementación de nuevas funcionalidades: modo supervisor, gestión de bajas, carga de fechas, y validación de transporte.
- Generación de la documentación completa (README, informe Word, diagrama BPMN).

Prompts utilizados durante el desarrollo:

- 'Corregí el error padStart por zfill'
- 'Hace el código más simple como si lo hubiera hecho un principiante'
- 'Las fechas deben borrarse del listado al ser seleccionadas'
- 'Agregá funciones al supervisor para cargar nuevas fechas'
- 'Si no tengo solicitud de horas no debería dejarme solicitar transporte'

7. Conclusión

La integración del chatbot utilizando la API oficial de Telegram y una persistencia local estructurada en archivos CSV demuestra un canal práctico y directo de comunicación corporativa que cumple con todos los objetivos del negocio y del modelado BPMN 2.0. La incorporación del carril del Supervisor agrega valor al flujo de aprobación, mientras que la máquina de estados garantiza que cada conversación mantenga coherencia en todo momento. Las validaciones de entrada (camino infeliz) y la gestión de fechas disponibles completan un sistema funcional que resuelve un problema administrativo real de manera automatizada.

8. Referencias

Object Management Group (OMG). (2011). Business Process Model and Notation (BPMN), Version 2.0.

Python Software Foundation. (2025). CSV File Reading and Writing. Python Standard Library.

Telegram Messenger LLP. (2025). Telegram Bot API Documentation.

OpenCode. (2025). AI-powered coding agent. <https://opencode.ai>